UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 2 (10%) – Transaction Management

Name	:	Tan Zhi Ming
Matric Id	:	A23CS0189
Section	:	01

Name	:	Ng Kuhin
Matric Id	:	A23CS0148
Section	:	01

Instructions:

This assignment evaluates your ability to apply the concept of serializability, differentiate between serial and non-serial transaction schedules and implement locking techniques to manage simultaneous transactions.

Academic Integrity

All students must work in a group of **TWO(2)**. The use of AI-generated tools or sharing of answers between students **is strictly prohibited** and will result in **zero marks**. You are required to complete the answers in **handwritten form**. Please take **clear screenshots** of your answers and submit them in **PDF format**.

QUESTION 1 (15 MARKS)

Consider the schedule transactions in Table 1 and answer all following questions (i) – (iv).

Table 1: Schedule T1, T2 and T3

Time	T1	T2	T3
t_1	begin transaction		
t_2	read_item(X)		
t_3	read_item(Y)	begin transaction	
t_4	commit	read_item(M)	begin transaction
t_5		read_item(N)	read_item(N)
t_6		M=(N+M)*10	read_item(M)
t_7		write_item(M)	N=N+150
t_8		commit	write_item(N)
t_9			commit

i) Draw the precedence graph for the transaction schedule in Table 1 to check for serializability.

ii) Explain the precedence graph produced in (i).

iii) Apply the locking technique to the transaction schedule in Table 1, and revise the transaction schedule in Table 1 using the following format:

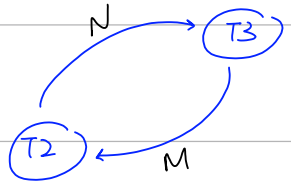
T1	T2	T3

iv) Identify the problem(s) that arise in (iii) and then explain any potential problems that may still occur after applying the locking technique in concurrency control based on your answer in (iii).

日期: /

(5)

i)



ii) Based on i) graph, there occur a cycle ($T_2 \rightarrow T_3$ and $T_3 \rightarrow T_2$)

- T3 writes to item N after T2 has read it ($T_2 \rightarrow T_3$)
- T2 writes to item M after T3 has read it ($T_3 \rightarrow T_2$)

∴ Since a cycle exists, the schedule is not conflict serializable

iii)

time	T_1	T_2	T_3
T_1	begin		
T_2	read_lock item(X)		
T_3	read (X)	begin	
T_4	read_lock item(Y)	write_lock (M)	begin
T_5	read (Y)	read (M)	write_lock (N)
T_6	commit	read_lock (N) (wait)	read (N)
T_7	unlock (X, Y)	wait	read_lock (M) wait
T_8		wait	wait
T_9		wait	wait
T_{10}		wait	wait
T_{11}			
T_{12}			
T_{13}			
T_{14}			

iv) Primary problem: Deadlock, T_2 and T_3 are stuck in a circular wait where each holds a resource the other needs.

Potential problem: Starvation, if T_2 was never granted its lock due to a stream of other transaction

QUESTION 2 (15 MARKS)

Based on the transaction schedule in Table 2, answer all the following questions:

Table 2: Transaction Schedule for T4-T5-T6

	T4	T5	T6
	begin transaction		
	read_item(P)		
	read_item(Q)	begin transaction	
↓	P=(P*5)+Q	read_item(Q)	
time	write_item(P)	read_item(R)	begin transaction
	commit	Q=Q+R	read_item(R)
		write_item(Q)	R=R-100
		commit	write_item(R)
			commit

i) Draw the precedence graph for the transaction schedule in Table 2 to check for serializability.

ii) Explain the precedence graph produced in (i).

iii) Apply the two-phase locking (2PL) protocol to emphasize the positioning of lock and unlock operations in each transaction using the following format.

T4	T5	T6

QUESTION 3 (25 MARKS)

a) Consider the schedule transactions in Table 3.1 and answer all following questions (i) – (iv).

Table 3.1: Schedule T7, T8, T9 and T10

Time	T7	T8	T9	T10
t_1	begin transaction	begin transaction		
t_2	read (X)	read (Y)	begin transaction	
t_3	X = X + 50		read (X)	begin transaction
t_4	write (X)			read (Z)
t_5		read (X)	X = X - 15	Z = Z + 20
t_6		X = Y * 12	write (X)	write (Z)
t_7		write (X)	commit	
t_8	commit	commit		commit

i) Is the schedule in Table 3.1 a serial or non-serial schedule? State your reason.

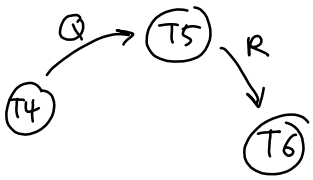
ii) Draw the precedence graph for the transactions given in Table 3.1.

iii) Does the schedule in Table 3.1 have conflict-serializable or not? How can you determine this?

iv) What problem does the schedule in Table 3.1 face?

(2)

i)



ii) The precedence graph is acyclic.
∴ the schedule is conflict-serializable, equivalent serial order is T4 → T5 → T6

iii)

Table 2: Transaction Schedule for T4-T5-T6

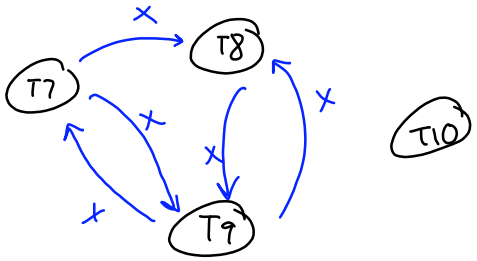
	T4	T5	T6
	begin transaction		
	read_item(P)		
	read_item(Q)	begin transaction	
↓	P=(P*5)+Q	read_item(Q)	
time	write_item(P)	read_item(R)	begin transaction
	commit	Q=Q+R	read_item(R)
		write_item(Q)	R=R-100
		commit	write_item(R)
			commit

time	T4	T5	T6
t_1	begin		
t_2	write_lock (P)		
t_3	read (P)	begin	
t_4	read_lock (Q)	write_lock (Q) wait	
t_5	read (Q)	w	begin
t_6	P = (P*5)+Q	w	write_lock (R)
t_7	write (P)	w	read (R)
t_8	commit / unlock (P, Q)	w	R = R-100
t_9		read (Q)	write (R)
t_{10}		read_lock (R) wait	commit / unlock (R)
t_{11}		read (R)	
t_{12}		Q = Q+R	
t_{13}		write (Q)	
t_{14}		commit / unlock (Q, R)	

Q3 a)

i) non-serial, the reason is operations from multiple transaction are interleaved over time

ii)



iii) No, the schedule is not conflict-serializability. As shown in the part ii), there are cycles (T_7 and T_9) and (T_9 and T_8)

iv) The schedule faces lost update. Both T_7 and T_9 read initial value of X at roughly same time (t_2 and t_3). T_7 updates X and writes it at t_4, but then T_9 calculate its own version of X and writes it at t_6, effectively overwriting the update made by T_7. It also faced dirty read problem, because T_8 reads the X at t_5 that was written by T_7, but T_7 does not commit until T_8

b) i)

Time	T11	T12	T13	T14
t_1	begin			
t_2	S-lock (A)	begin		
t_3	read (A)	S-lock (B)	begin	
t_4	A= A+200	read (B)	S-lock (A) queue	begin
t_5	unlock (A)	B= B-50	wait	S-lock (C)
t_6	X-lock (A) queue	unlock (B)	S-lock (A) granted	read (C)
t_7	wait	X-lock (B)	read (A)	unlock (C)
t_8	wait	write (B)	A= A-30	C= C+10
t_9	wait	unlock (B)	unlock (A)	X-lock (C)
t_{10}	X-lock (A) granted	S-lock (A) queue	X-lock (A) queue	write (C)
t_{11}	write (A)	wait	wait	
t_{12}	commit / unlock (A)	wait	wait	commit / unlock(C)
t_{13}		S-lock (A) granted	wait	
t_{14}		read (A)	wait	
t_{15}		A = A * 2	wait	
t_{16}		unlock (A)	wait	
t_{17}		X-lock (A) queue	X-lock (A) granted	
t_{18}		wait	write (A)	
t_{19}		wait	commit / unlock (A)	
t_{20}		X-lock (A) granted		
t_{21}		write (A)		
t_{22}		commit / unlock (A)		

ii) Lost update problem, based on the schedule i) both T_{11} and T_{13} read the initial value of A (t_3 and t_6) before either of them perform write operation.

- T_{11} update A at t_{11}
- T_{13} update A at t_{18}

Because T_{13} overwrites the value written by T_{11} without seeing T_{11}'s changes, T_{11}'s update lost.

iii)

Time	T_{11}	T_{12}	T_{13}	T_{14}
t_1	begin			
t_2	X-lock (A)	begin		
t_3	read (A)	X-lock (B)	begin	
t_4	A= A+200	read (B)	X-lock(A) queue	begin
t_5	write (A)	B= B-50	wait	X-lock (C)
t_6	commit / unlock (A)	write (B)	wait	read (c)
t_7			X-lock (A) granted	C=C+10
t_8			read (A)	write (c)
t_9		X-lock (A) queue	A=A-30	
t_{10}		wait	write (A)	
t_{11}		wait	commit / unlock (A)	
t_{12}		X-lock (A) granted		commit/unlock(C)
t_{13}		read (A)		
t_{14}		A= A*2		
t_{15}		write (A)		
t_1		commit /unlock (B,A)		

iv) Issues : deadlock. The schedule creates a "Hold and Wait" situation. At t_9, T12 holds a lock on B while waiting for A. If T_{13} had needed to access B, a deadlock would have occured.

Reduced concurrency: The strict locking protocol cause significant blocking. T_{12} is forced to wait from t_9 to t_{11}. extending its completion time significantly compared to original schedule

b) Consider the schedule transactions in Table 3.2 and answer all following questions (i) – (iv).

Table 3.2: Schedule T11, T12, T13 and T14

Time	T11	T12	T13	T14
t_1	begin transaction			
t_2	read (A)	begin transaction		
t_3		read (B)	begin transaction	
t_4	A = A + 200	B = B - 50	read (A)	begin transaction
t_5	write (A)			read (C)
t_6		write (B)	A = A - 30	C = C + 10
t_7			write (A)	write (C)
t_8	commit			
t_9		read (A)		
t_{10}		A = A * 2		
t_{11}		write (A)		
t_{12}		commit	commit	commit

i) Implement the basic locking to the schedule in Table 3.2 by specifying the appropriate lock type (shared or exclusive) for each transaction.

ii) Based on your answer in (i), state any other issues that may arise.

iii) Implement the Two-Phase Locking (2PL) protocol to the schedule in Table 3.2.

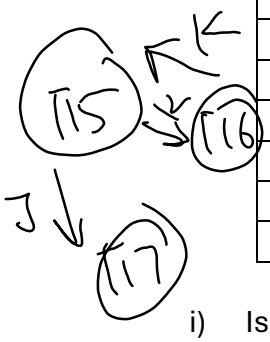
iv) Based on your answer in (iii), state any other issues that may arise.

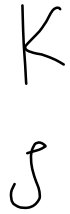
QUESTION 4 (15 MARKS)

Consider the schedule transactions in Table 4 and answer all following questions (i) – (v).

Table 4.1: Schedule T15, T16 and T17

Time	T15	T16	T17
t_1	begin transaction		
t_2	read (J)	begin transaction	
t_3	J = J + 10	read (K)	begin transaction
t_4	write (J)	K = K - 5	read (J)
t_5	Read (K)	write (K)	J = J - 15
t_6	K = K + 20		write (J)
t_7	write (K)	commit	
t_8	commit		commit





i) Is the schedule in Table 4 a serial or non-serial schedule? State your reason.

ii) Draw the precedence graph for the transactions given in Table 4.

iii) Does the schedule in Table 4 have conflict-serializable or not? How can you determine this?

iv) What problem does the schedule in Table 4 face?

v) Apply the two-phase locking (2PL) protocol to emphasize the positioning of lock and unlock operations in each transaction using the following format.

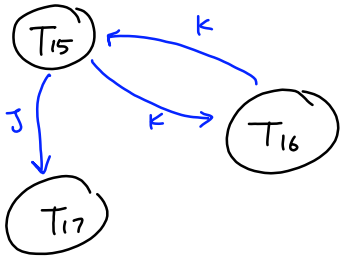
Submission

1. Submission due date: **24 December 2025 (before 10:00PM)**.
2. Each group must submit only **ONE** answer in PDF format.
3. All work must be original and completed independently. The use of AI-generated tools or sharing of answers between students is strictly prohibited.
4. Late submissions will be penalized according to faculty policy

Q4

i) non-Serial schedule. The operation of the transaction (T_{15}, T_{16}, T_{17}) are interleaved (mixed together)

ii)



iii) No conflict - Serializable The precedence graph contains a cycle.

iv) Proble:

- Lost Update Problem (item K)
  when T_{15} write at t_7, it overwrites T_{16}'s update without considering it. T_{16}'s work is lost.
- Dirty Read Problem (item J)
  Since T_{15} has not committed yet, T_{17} is reading uncommited dirty data.
  If T_{15} rollback, T_{17} would working with invalid data.

Time	T_{15}	T_{16}	T_{17}
t_1	begin		
t_2	X-lock (J)	begin	
t_3	read (J)	X-lock (K)	begin
t_4	J = J+10	read (K)	X-lock (J) queue
t_5	write (J)	K = K-5	wait
t_6	X-lock (K) queue	write (K)	wait
t_7	wait	commit / unlock (K)	wait
t_8	X-lock (K) granted		wait
t_9	read (K)		wait
t_{10}	K=K +20		wait
t_{11}	write (K)		wait
t_{12}	commit / unlock (J,K)		wait
t_{13}			X-lock (J) granted
t_{14}			read (J)
t_{15}			J = J-15
t_{16}			write (J)
t_{17}			commit / unlock (J)

UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

SECP3623
Database Programming
2025/2026 SEMESTER 1

Assignment 2 (10%) – Transaction Management

Name	:	
Matric Id	:	
Section	:	

Name	:	
Matric Id	:	
Section	:	

Instructions:

This assignment evaluates your ability to apply the concept of serializability, differentiate between serial and non-serial transaction schedules and implement locking techniques to manage simultaneous transactions.

Academic Integrity

All students must work in a group of **TWO(2)**. The use of AI-generated tools or sharing of answers between students **is strictly prohibited** and will result in **zero marks**. You are required to complete the answers in **handwritten form**. Please take **clear screenshots** of your answers and submit them in **PDF format**.

QUESTION 1 (15 MARKS)

Consider the schedule transactions in Table 1 and answer all following questions (i) – (iv).

Table 1: Schedule T1, T2 and T3

Time	T1	T2	T3
t_1	begin transaction		
t_2	read_item(X)		
t_3	read_item(Y)	begin transaction	
t_4	commit	read_item(M)	begin transaction
t_5		read_item(N)	read_item(N)
t_6		M=(N+M)*10	read_item(M)
t_7		write_item(M)	N=N+150
t_8		commit	write_item(N)
t_9			commit

i) Draw the precedence graph for the transaction schedule in Table 1 to check for serializability.

ii) Explain the precedence graph produced in (i).

iii) Apply the locking technique to the transaction schedule in Table 1, and revise the transaction schedule in Table 1 using the following format:

T1	T2	T3

iv) Identify the problem(s) that arise in (iii) and then explain any potential problems that may still occur after applying the locking technique in concurrency control based on your answer in (iii).

Q1

i.

ii. T_1 only accesses item X & Y
T_2 writes after T_3 reads item M. An edge from T_3 to T_2
T_3 writes after T_2 reads item N. An edge from T_2 to T_3
Precedence graph has a cycle
It is a Non-Conflict Serializable Schedule

iii.

	T_1	T_2	T_3
t_1	begin transaction		
t_2	read_lock (X)		
t_3	read_item(X)	begin transaction	
t_4	unlock(X)	write_lock(M)	begin transaction
t_5	read_lock(Y)	read_item(M)	write_lock(N)
t_6	read_item(Y)	read_lock(N)	read_item(N);
t_7	commit; unlock(Y)	WAIT	read_lock(M)
t_8		WAIT	WAIT
t_9		—	—

iv. Deadlock, Neither T_2 nor T_3 can proceed because each is waiting for other to release a resource which are N and M. where the system hangs indefinitely

The potential problem that may occur is Starvation.
If the deadlock was ever to be solved, either one transaction must abort in order to let the other proceed.
Example: T2 aborts, T_3 is able to proceed but T_2 will need to wait if there are many locks in T_3, and leads to perpetually delayed

QUESTION 2 (15 MARKS)

Based on the transaction schedule in Table 2, answer all the following questions:

Table 2: Transaction Schedule for T4-T5-T6

↓ time	T4	T5	T6
	begin transaction		
	read_item(P)		
	read_item(Q)	begin transaction	
	P=(P*5)+Q	read_item(Q)	
	write_item(P)	read_item(R)	begin transaction
	commit	Q=Q+R	read_item(R)
		write_item(Q)	R=R-100
		commit	write_item(R)
			commit

i) Draw the precedence graph for the transaction schedule in Table 2 to check for serializability.

ii) Explain the precedence graph produced in (i).

iii) Apply the two-phase locking (2PL) protocol to emphasize the positioning of lock and unlock operations in each transaction using the following format.

T4	T5	T6

QUESTION 3 (25 MARKS)

a) Consider the schedule transactions in Table 3.1 and answer all following questions (i) - (iv).

Table 3.1: Schedule T7, T8, TG and T10

Time	T7	T8	TG	T10
t_1	begin transaction	begin transaction		
t_2	read (X)	read (Y)	begin transaction	
t_3	X = X + 50		read (X)	begin transaction
t_4	write (X)			read (Z)
t_5		read (X)	X = X - 15	Z = Z + 20
t_6		X = Y * 12	write (X)	write (Z)
t_7		write (X)	commit	
t_8	commit	commit		commit

i) Is the schedule in Table 3.1 a serial or non-serial schedule? State your reason.

ii) Draw the precedence graph for the transactions given in Table 3.1.

iii) Does the schedule in Table 3.1 have conflict-serializable or not? How can you determine this?

iv) What problem does the schedule in Table 3.1 face?

Q2

i.

ii. T_5 writes after T_4 reads item Q. An edge from T_4 to T_5
T_6 writes after T_5 reads item R. An edge from T_5 to T_6
Precedence graph has no cycle
It is a Conflict Serializable Schedule

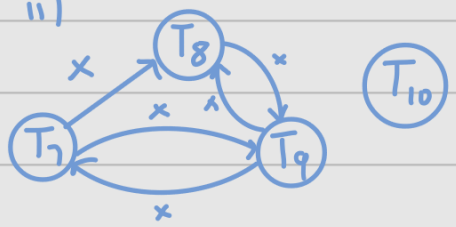
iii.

	T_4	T_5	T_6
t_1	begin transaction		
t_2	write_lock(P)		
t_3	read_item(P)	begin transaction	
t_4	read_lock(Q)	write_lock(Q)	
t_5	read_item(Q)	WAIT	begin transaction
t_6	P=(P+5)+Q	WAIT	write_lock(R)
t_7	write_item(P)	WAIT	read_item(R)
t_8	commit/unlock(P,Q)	WAIT	R=R-100
t_9		read_item(Q)	write_item(R)
t_{10}		WAIT	commit/unlock(R)
t_{11}		read_lock(R)	
t_{12}		read_item(R)	
t_{13}		Q=Q+R	
t_{14}		write_item(Q)	
t_{15}		commit;unlock(Q,R)	
t_{16}			
t_{17}			
t_{18}			

Q3 a)

i) Non-serial schedule

ii)



iii) The precedence graph contains cycles.
It is a Non-Conflict Serializable Schedule

iv) Lost Update: Both T_9 & T_8 overwrite data.
T_9 overwrites T_8 at t_6
T_8 overwrites T_9 at t_7

b) Consider the schedule transactions in Table 3.2 and answer all following questions (i) – (iv).

Table 3.2: Schedule T11, T12, T13 and T14

Time	T11	T12	T13	T14
t_1	begin transaction			
t_2	read (A)	begin transaction		
t_3		read (B)	begin transaction	
t_4	A = A + 200	B = B - 50	read (A)	begin transaction
t_5	write (A)			read (C)
t_6		write (B)	A = A - 30	C = C + 10
t_7			write (A)	write (C)
t_8	commit			
t_9		read (A)		
t_{10}		A = A * 2		
t_{11}		write (A)		
t_{12}		commit	commit	commit

i) Implement the basic locking to the schedule in Table 3.2 by specifying the appropriate lock type (shared or exclusive) for each transaction.

ii) Based on your answer in (i), state any other issues that may arise.

iii) Implement the Two-Phase Locking (2PL) protocol to the schedule in Table 3.2.

iv) Based on your answer in (iii), state any other issues that may arise.

Q3 b) i)

	T_{11}	T_{12}	T_{13}	T_{14}
t_1	begin transaction			
t_2	lock_S (A)	begin transaction		
t_3	read (A)	lock_X (B)	begin transaction	
t_4	unlock (A)	read (B)	lock_S (A)	begin transaction
t_5	lock_X (A)	B = B - 50	read (A)	lock_X (C)
t_6	WAIT		unlock (A)	read (C)
t_7	A = A + 200	write (B)	lock_X (A)	C = C + 10
t_8	write (A)	unlock (B)	WAIT	write (C)
t_9	unlock (A)	lock_X (A)	WAIT	unlock (C)
t_{10}		WAIT	A = A - 30	
t_{11}		WAIT	write (A)	
t_{12}	commit	WAIT	unlock (A)	
t_{13}		read (A)		
t_{14}		A = A * 2		commit
t_{15}		write (A)		
t_{16}		commit		
t_{17}			commit	

ii) Lost Update : T_{13} is overwritten by T_{12} in T_{15}

Uncommitted Dependency : T_{13} has not yet committed before T_{12} has already committed

iii)

	T_{11}	T_{12}	T_{13}	T_{14}
t_1	begin transaction			
t_2	lock_X(A)	begin transaction		
t_3	read (A)	lock_X(B)	begin transaction	
t_4		read(B)	lock_X(A)	begin transaction
t_5	A=A+200	B=B-50	WAIT	lock_X(C)
t_6	write (A)		WAIT	read(C)
t_7		write(B)	WAIT	C=C+10
t_8			WAIT	write(C)
t_9	commit/unlock(A)		WAIT	
t_{10}		lock_X(A)	read(A)	
t_{11}		WAIT		
t_{12}		WAIT	A=A-30	
t_{13}		WAIT	Write (A)	commit/unlock(C)
t_{14}		WAIT		
t_{15}		WAIT		
t_{16}		WAIT		
t_{17}		WAIT		
t_{18}		WAIT	commit/unlock(A)	
t_{19}		read(A)		
t_{20}		A=A*2		
t_{21}		write(A)		
t_{22}		commit/unlock(A/B)		

iv) Deadlock

QUESTION 4 (15 MARKS)

Consider the schedule transactions in Table 4 and answer all following questions (i) - (v).

Table 4.1: Schedule T15, T16 and T17

Time	T15	T16	T17
t_1	begin transaction		
t_2	read (J)	begin transaction	
t_3	J = J + 10	read (K)	begin transaction
t_4	write (J)	K = K - 5	read (J)
t_5	Read (K)	write (K)	J = J - 15
t_6	K = K + 20		write (J)
t_7	write (K)	commit	
t_8	commit		commit

i) Is the schedule in Table 4 a serial or non-serial schedule? State your reason.

ii) Draw the precedence graph for the transactions given in Table 4.

iii) Does the schedule in Table 4 have conflict-serializable or not? How can you determine this?

iv) What problem does the schedule in Table 4 face?

v) Apply the two-phase locking (2PL) protocol to emphasize the positioning of lock and unlock operations in each transaction using the following format.

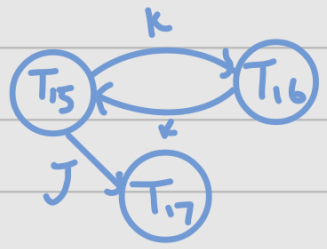
Submission

1. Submission due date: **24 December 2025 (before 10:00PM)**.
2. Each group must submit only **ONE** answer in PDF format.
3. All work must be original and completed independently. The use of AI-generated tools or sharing of answers between students is strictly prohibited.
4. Late submissions will be penalized according to faculty policy

Q4

i) Non-serial schedule

ii)



iii) Non Conflict Serializable Schedule
Precedence graph has a cycle

iv) Uncommitted Dependency

	T_{11}	T_{12}	T_{13}
t_1	begin transaction		
t_2	lock_X(J)	begin transaction	
t_3	read(J)	lock_X(K)	begin transaction
t_4	J=J+10	read(K)	lock_X(J)
t_5	write(J)	K=K-5	
t_6	lock_X(K)	write(K)	
t_7			
t_8		commit/unlock(K)	
t_9	read(K)		
t_{10}	K=K+10		
t_{11}	write(K)		
t_{12}	commit/unlock(J/K)		
t_{13}			read(J)
t_{14}			J=J-15
t_{15}			write(J)
t_{16}			
t_{17}			commit/unlock(J)
t_{18}			